AI23531 – DEEP LEARNING

EX 8 IMAGE GENERATION USING A VARIATIONAL AUTOENCODER (VAE)

AIM

To implement a Variational Autoencoder that learns the latent representation of face images (CelebA) and generates new images from the learned distribution.

PROCEDURE

1. Import PyTorch and torchvision; set image size 64, latent dimension 100, 5 epochs.
2. Load a 500-image subset of CelebA, centre-crop, resize and normalise it.
3. Define the Encoder (4 convolution layers producing mean and log-variance), the Decoder (4 transposed-convolution layers) and the VAE with the reparameterisation trick.
4. Define the loss as reconstruction (MSE) loss plus KL divergence.
5. Train with Adam (lr = 1e-3) and print the loss each epoch.
6. After every epoch, sample random latent vectors, decode them and display a grid of generated faces.

ALGORITHM

1. Start.
2. Load and preprocess the CelebA images.
3. Encode each image to μ and $\log \sigma^2$.
4. Sample $z = \mu + \varepsilon \cdot \sigma$ with $\varepsilon \sim N(0, I)$ (reparameterisation).
5. Decode z to reconstruct the image.
6. Compute loss = MSE(reconstruction, input) + KL divergence; backpropagate and update with Adam.
7. After each epoch, sample $z \sim N(0, I)$, decode it and display a 64-image grid.
8. Repeat for all epochs.
9. Stop.

PROGRAM

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
import matplotlib.pyplot as plt

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
image_size = 64
batch_size = 128
latent_dim = 100
num_epochs = 5
learning_rate = 1e-3

transform = transforms.Compose([
```

```
    transforms.CenterCrop(178),
    transforms.Resize(image_size),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
dataset = datasets.CelebA(root='data', split='train', download=True,
transform=transform)
dataset = Subset(dataset, range(500))
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

class Encoder(nn.Module):
    def __init__(self, latent_dim):
        super(Encoder, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1), nn.ReLU(),
            nn.Conv2d(64, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.Conv2d(128, 256, 4, 2, 1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.Conv2d(256, 512, 4, 2, 1), nn.BatchNorm2d(512), nn.ReLU()
        )
        self.fc_mu = nn.Linear(512*4*4, latent_dim)
        self.fc_logvar = nn.Linear(512*4*4, latent_dim)
    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        return self.fc_mu(x), self.fc_logvar(x)

class Decoder(nn.Module):
    def __init__(self, latent_dim):
        super(Decoder, self).__init__()
        self.fc = nn.Linear(latent_dim, 512*4*4)
        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(512, 256, 4, 2, 1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.ConvTranspose2d(256, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.ConvTranspose2d(128, 64, 4, 2, 1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.ConvTranspose2d(64, 3, 4, 2, 1), nn.Tanh()
        )
    def forward(self, z):
        x = self.fc(z)
        x = x.view(x.size(0), 512, 4, 4)
        return self.deconv(x)

class VAE(nn.Module):
    def __init__(self, latent_dim):
        super(VAE, self).__init__()
        self.encoder = Encoder(latent_dim)
        self.decoder = Decoder(latent_dim)
    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std
    def forward(self, x):
        mu, logvar = self.encoder(x)
        z = self.reparameterize(mu, logvar)
        return self.decoder(z), mu, logvar

def vae_loss(recon_x, x, mu, logvar):
    recon_loss = F.mse_loss(recon_x, x, reduction='sum')
    kl_div = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon_loss + kl_div

model = VAE(latent_dim).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    for images, _ in dataloader:
```

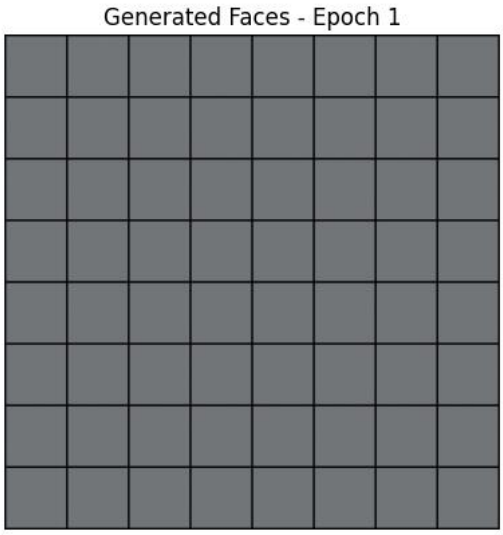
```
        images = images.to(device)
        recon, mu, logvar = model(images)
        loss = vae_loss(recon, images, mu, logvar)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch [{epoch+1}/{num_epochs}], Loss:
{total_loss/len(dataloader.dataset):.4f}")

    model.eval()
    with torch.no_grad():
        z = torch.randn(64, latent_dim).to(device)
        sample_images = model.decoder(z).cpu() * 0.5 + 0.5
        grid = torchvision.utils.make_grid(sample_images, nrow=8)
        plt.imshow(grid.permute(1, 2, 0))
        plt.axis('off')
        plt.title(f"Generated Faces - Epoch {epoch+1}")
        plt.show()
```

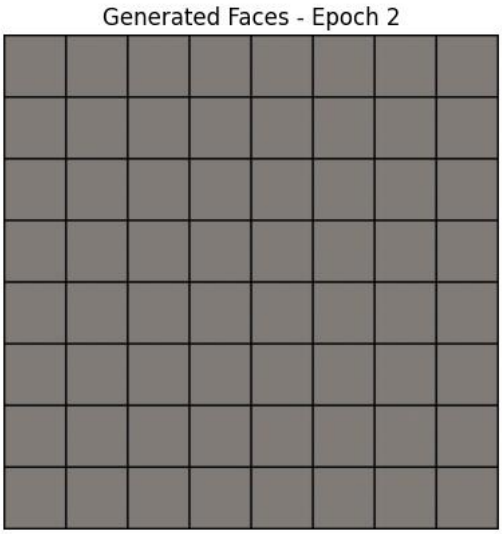
OUTPUT

```
Epoch [1/5], Loss: 149561758.6764
Epoch [2/5], Loss: 4215.5495
Epoch [3/5], Loss: 3495.4728
Epoch [4/5], Loss: 3206.1232
Epoch [5/5], Loss: 2963.3496
```

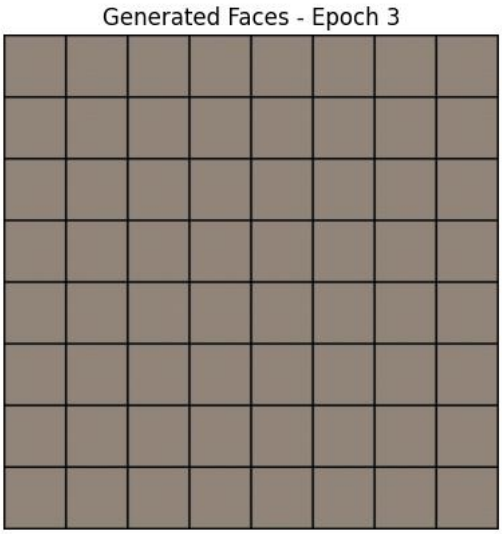
Console output



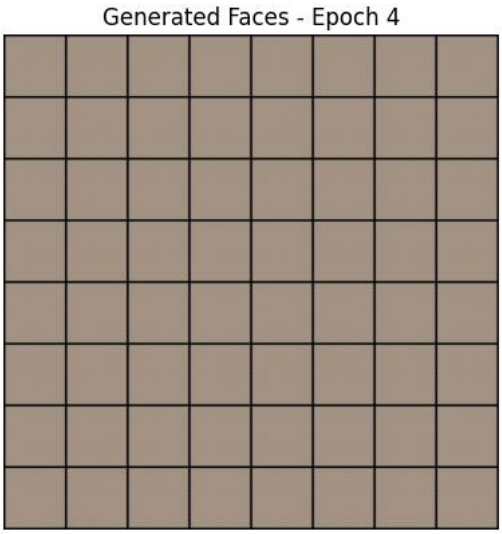
Generated faces – Epoch 1



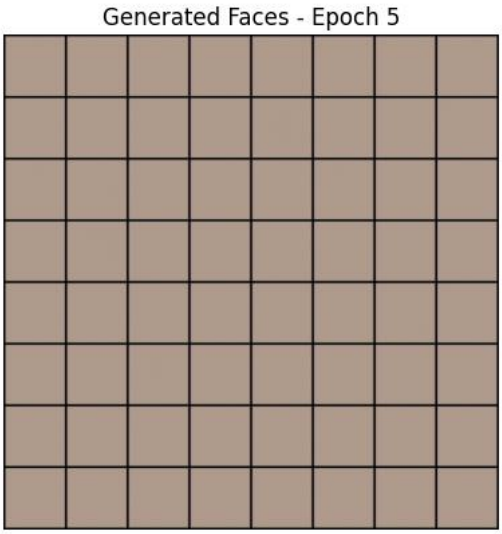
Generated faces – Epoch 2



Generated faces – Epoch 3



Generated faces – Epoch 4



Generated faces – Epoch 5

RESULT

The loss dropped from 149,561,758.68 in epoch 1 to 2,963.35 in epoch 5, and the generated face grids became progressively more structured, although they remain blurry because only 500 images were used.